9-Hour Hands-On Java Training: Flight Reservation System

For your second-year UG Computer Science students, I recommend developing a **console-based Flight Reservation System using Java 8+**, with the entire project built step by step during the training.

The project will help students understand Functional Interfaces, how they differ from regular interfaces, and how the Stream API simplifies searching, filtering, sorting, grouping, and analysing flight bookings.

The important teaching approach is to build a working application first, then progressively improve it using functional programming features rather than teaching the concepts only through isolated examples.

1. Project overview







Project title: Flight Reservation System — Efficient Airline Booking and Analytics

Scenario: An airline company needs a system that manages flights, passenger bookings, seat availability, ticket cancellation, and booking reports efficiently, even when the number of flights and bookings increases.

Functional requirements

Module	Features
Flight management	View flights, search by source and destination

Flight search	Filter by destination, price, date, or availability
Reservation	Book tickets and generate booking IDs
Cancellation	Cancel reservations and restore seat availability
Booking management	Search bookings by ID or passenger
Analytics	Calculate revenue, group bookings by destination, find popular flights
Functional programming	Use custom functional interfaces, built-in interfaces, lambdas, method references, and streams

For this 9-hour training, the project will use in-memory collections instead of a database. That keeps the focus on Java programming concepts. Data will reset when the application restarts; database persistence can be a later extension.

2. Mapping your syllabus to the project

Functional interfaces

`Predicate`, `Function`, `Consumer`, `Supplier`, custom `FlightFilter`, and `@FunctionalInterface`.

Project application: validate booking conditions, filter eligible flights, calculate ticket prices, and generate booking IDs.

Comparison of regular interfaces and functional interfaces

Abstract methods, default methods, static methods, lambda compatibility, and anonymous classes.

Project application: compare traditional flight-search implementations with lambda-based filters and sorting strategies.

Stream API

`filter()`, `map()`, `sorted()`, `distinct()`, `limit()`, `collect()`, `count()`, `reduce()`, and `Collectors.groupingBy()`.

Project application: search available flights, rank by fare, calculate revenue, and generate booking analytics.

3. Nine-hour training plan

This schedule assumes nine instructional hours, with practical coding throughout. It can be delivered over three days, with three hours per day.

Session	Duration	Teaching and development activity
1	1 hour	Project requirements, classes, objects, collections, and traditional interfaces

2	1 hour	Functional interfaces, `@FunctionalInterface`, lambda expressions
3	1 hour	`Predicate`, `Function`, `Consumer`, `Supplier` and custom functional interfaces
4	1 hour	Flight model, sample data, and flight listing
5	1 hour	Search and filter flights using lambdas and `Predicate`
6	1 hour	Reservation, seat validation, cancellation, and booking management
7	1 hour	Stream operations: `filter`, `map`, `sorted`, `count`, `reduce`
8	1 hour	Revenue reports, grouping, fare sorting, and booking analytics
9	1 hour	Integration, testing, debugging, and student practical assessment

Trainer recommendation: Complete the initial flight model and booking workflow before introducing streams. Then refactor the traditional search and report logic into functional-style implementations. This makes the advantages visible to students.

4. Complete Java project source code

To make the first practical session manageable, the following version uses a **single Java source file**. Students can run it without Maven, a database, or external libraries.

FlightReservationSystem.java

Java 8 or later · Console application · In-memory storage

Includes flight search, booking, cancellation, functional interfaces, lambdas, streams, and reports.

Step 1 — Create the project

In Eclipse:

1. Create a Java Project named `FlightReservationSystem`.
2. Select JDK 8 or a later JDK.
3. Under `src`, create a class named `FlightReservationSystem`.
4. Select the option to create a `main()` method.
5. Replace the generated code with the complete source below.

Step 2 — Complete source code

Java

```
import java.util.*;
import java.util.function.*;
```

```
import java.util.stream.Collectors;

public class FlightReservationSystem {

    // 1. Flight model
    static class Flight {
        private final String flightNo;
        private final String airline;
        private final String source;
        private final String destination;
        private final double fare;
        private int availableSeats;

        Flight(String flightNo, String airline, String source,
               String destination, double fare, int seats) {
            this.flightNo = flightNo;
            this.airline = airline;
```

Step 3 — Compile and run

From the folder containing `FlightReservationSystem.java`, use:

Bash

```
javac FlightReservationSystem.java
java FlightReservationSystem
```

In Eclipse, right-click the class and select **Run As → Java Application**.

Note for students: this is a teaching prototype, not a production airline reservation platform. It has no database, payment gateway, authentication, or concurrent booking protection. A production system would need transactional persistence and concurrency control to prevent two users from reserving the same final seat.

5. Teaching the comparison: regular interface vs functional interface

Use this small example before students begin the main project.

Example A — Traditional interface implementation

Java

```
interface FlightSearch {
    void search();
```

```
}

class DelhiFlightSearch implements FlightSearch {
    @Override
    public void search() {
        System.out.println("Searching flights to Delhi");
    }
}

// Usage
FlightSearch search = new DelhiFlightSearch();
search.search();
```

Here, the class explicitly implements the interface.

Example B — Functional interface implementation

Java

```
@FunctionalInterface
interface FlightSearch {
    void search();
}

// Usage with lambda
FlightSearch search =
    () -> System.out.println("Searching flights to Delhi");

search.search();
```

The lambda expression implements the interface's single abstract method without requiring a separate implementation class.

Comparison table

Feature	Regular interface	Functional interface
Abstract methods	Can declare multiple	Exactly one
Default methods	Allowed	Allowed
Static methods	Allowed	Allowed
Lambda expression	Not directly compatible if multiple abstract methods exist	Compatible

Main purpose	Define a contract for implementations	Enable lambdas and functional-style programming
Project example	Booking service contract	Flight filtering strategy

An interface with one abstract method is a functional interface whether or not it is annotated. `@FunctionalInterface` makes the compiler verify this requirement. Methods inherited from `Object` do not count as additional abstract methods for this rule.

6. Important project demonstrations for each concept

These examples can be run individually during your training.

A. Custom functional interface with lambda

Java

```
FlightFilter lowFareFilter =
    flight -> flight.getFare() < 5000;

FlightReservationSystem system =
    new FlightReservationSystem();

system.searchFlights(lowFareFilter, "Flights below Rs.5000");
```

Explain: The `FlightFilter` interface has one abstract method, `test()`. The lambda supplies the method implementation.

B. Using `Predicate` and combining conditions

Java

```
Predicate<FlightReservationSystem.Flight> destination =
    f -> f.getDestination().equalsIgnoreCase("Delhi");

Predicate<FlightReservationSystem.Flight> affordable =
    f -> f.getFare() <= 6000;

Predicate<FlightReservationSystem.Flight> available =
    f -> f.getAvailableSeats() > 0;

Predicate<FlightReservationSystem.Flight> finalFilter =
    destination.and(affordable).and(available);
```

Explain: `Predicate<T>` accepts a value and returns `boolean`. Its `and()`, `or()`, and `negate()` methods allow conditions to be combined.

C. Using `Function` to transform flight information

Java

```
Function<FlightReservationSystem.Flight, String> summary =
    f -> f.getFlightNo() + " : " + f.getDestination()
        + " : Rs." + f.getFare();
```

Explain: `Function<T,R>` transforms a value of type `T` into a result of type `R`.

D. Using `Stream` to filter and sort flights

Java

```
system.flights.stream()
    .filter(f -> f.getDestination().equalsIgnoreCase("Delhi"))
    .filter(f -> f.getAvailableSeats() > 0)
    .sorted(Comparator.comparingDouble(
        FlightReservationSystem.Flight::getFare))
    .forEach(System.out::println);
```

Explain each operation:

- `stream()` creates a stream from the flight collection.
- `filter()` selects flights matching a condition.
- `sorted()` orders the matching flights by fare.
- `forEach()` displays the results.
- `Flight::getFare` is a method reference.

E. Revenue calculation with `mapToDouble()` and `sum()`

Java

```
double revenue = system.bookings.stream()
    .filter(FlightReservationSystem.Booking::isActive)
    .mapToDouble(FlightReservationSystem.Booking::getAmount)
    .sum();
```

```
System.out.println("Revenue = Rs." + revenue);
```

This illustrates a complete stream pipeline: filter active bookings, extract numerical amounts, and calculate the total.

7. Student hands-on activities

Practical completion tracker 0 of 6

- [ ] **Activity 1: Fare filter**
  Create a custom functional interface to select flights below a specified fare.
- [ ] **Activity 2: Destination search**
  Use Predicate and its and() method to select available flights for a destination.
- [ ] **Activity 3: Cheapest flights**
  Use filter(), sorted(), and limit() to display the three cheapest available flights.
- [ ] **Activity 4: Booking cancellation**
  Book two tickets, cancel one, and verify that seat availability and revenue are updated.
- [ ] **Activity 5: Revenue analytics**
  Calculate active booking revenue using mapToDouble() and sum().
- [ ] **Activity 6: Destination report**
  Use Collectors.groupingBy() and counting() to report the number of flights per destination.

8. Final assessment for the ninth hour

Ask students to extend the project independently:

1. Add a new flight to the collection.
2. Implement a custom functional interface for checking booking eligibility.
3. Search flights using two or more `Predicate` conditions.
4. Display the five cheapest available flights using streams.
5. Calculate revenue from confirmed bookings only.
6. Group confirmed bookings by airline or destination.
7. Demonstrate a method reference and explain how it differs from a lambda expression.
8. Test invalid flight numbers, unavailable seats, duplicate cancellation attempts, and empty passenger names.

Expected learning outcome: By the end of the nine hours, students should be able to explain functional interfaces, compare them with ordinary interfaces, implement lambda expressions, and use the Java Stream API to build meaningful features in a working Flight Reservation System.

One important distinction to teach: **streams do not automatically make every operation faster.** Their main benefit here is clearer, composable data-processing logic. Efficient high-volume booking also requires appropriate data structures, database indexing, transactions, and concurrency control—topics suitable for a follow-up advanced project.